# Activité 2 : Débruitage/réhaussement de la parole

# (Fourier, Soustraction spectrale)

*[Séance en « salle » machine 3h + rendu]*

**Objectifs :** Réaliser une analyse OLA d'un signal de parole par Fourier (spectre d'amplitude, spectre de phase, reconstruction du signal) et réaliser un algorithme de débruitage sur le principe de la soustraction spectrale.

**Outils :** Python (et Winsnoori pour lire et visualiser le signal modifié).

Le but du TP est de réaliser un traitement sur le signal de parole dans le domaine fréquentiel. L'entrée est un signal de parole, la sortie doit être un signal de parole.
**Au fur et à mesure du TP, nous ajouterons des étapes de traitement <u>entre</u> l'entrée et la sortie.**
Pour tester le programme avant les deux dernières étapes (6. Estimation du bruit et 7. Débruitage par soustraction spectrale), vous utiliserez le fichier test_seg.wav. Ensuite, pour l'estimation du bruit et le débruitage, vous utiliserez les fichiers *_bruit_ndB.wav avec n le niveau signal sur bruit (SNR – signal noise ratio) en décibels.

Pour appliquer différentes fonctions de traitement du signal nous utiliserons la librairie Librosa (assez utilisée en traitement du signal)

[`conda install -c conda-forge librosa` ou `pip install librosa`].

**Les pages d'aide pour librosa : https://librosa.org/doc/latest/index.html**

## 1. Récupération du signal [comme pour le TD1]

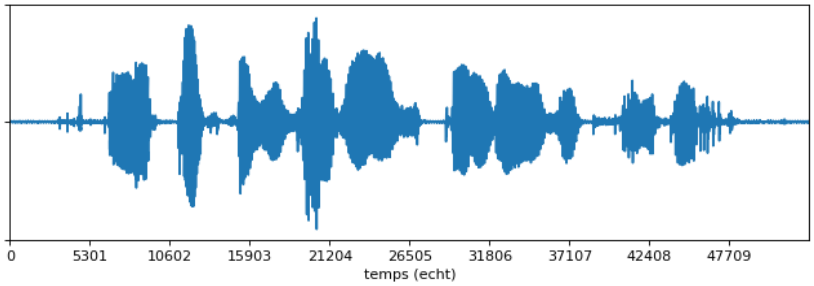
**Ouvrez un fichier** wav (et affichez l'information de la fréquence d'échantillonnage, et la taille du fichier en échantillons et en millisecondes)

| |
|---|
| Pour lire un fichier wav : **import librosa**<br>La fonction **librosa.load** renvoie le tableau des valeurs du signal et la fréquence d'échantillonnage. |

*[Rappel : si le signal est échantillonné à 22050 Hz : un signal de N échantillons correspond à N*1000/22050 ms et à l'inverse M ms correspond à M*22050/1000]*

**Réalisez une fonction d'affichage du signal :**

Pour afficher des points : **import matplotlib.pyplot as plt**
avec la fonction plot(signal) et show()
Il est possible de modifier les éléments du graphique : xticks pour les étiquettes de l'axe des x et xlabel pour le titre de l'axe de x (idem pour y).




**Quelques ajustements :**

La fonction **load** fournit des valeurs entre -1 et 1 et pour les traitements, le signal doit être sur 16 bits c'est-à-dire entre –32768 et + 32768 (2**15). Multipliez le signal pour avoir ces valeurs.

Enfin pour les traitements soient corrects, la moyenne du signal doit être nulle (cela recentre le signal sur 0 et on dit que l'on enlève la *composante continue* apportée par le micro). Calculez la moyenne du signal et soustrayez cette moyenne sur tout le tableau signal.

Réaffichez le signal pour vous assurer que les changements ont eu lieu.

## 2. Principe d'*OverLap and Add* et Fourier

Le principe est de calculer des spectres tous les 8 ms, sur un morceau du signal de 32ms. On utilisera la fenêtre de Hamming

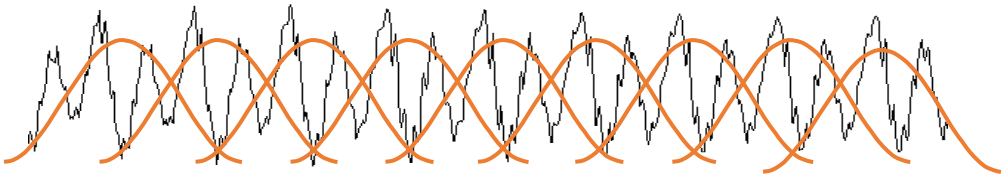


*Figure 1: L'écart entre 2 fenêtres est de m échantillons. Une fenêtre est de taille N.*

Une fonction de Librosa permet de le faire :
Regarder la documentation pour la fonction librosa.stft (Short Time Fourier Transform).
Et utiliser les paramètres suivants :
Hop size (taille du décalage des fenêtres) : 8ms (à convertir en echantillons)
Window size : 32 ms (idem)
Windows : hamming
Nombre de coefficient pour la FFT : 1024

**Enfin, nous allons reconstruire le signal** tout de suite pour vérifier que nous n'avons rien « cassé » dans le signal :

Dans un nouveau tableau signal_modif, utilisez la fonction **librosa.istft** (inverse STFT), avec les bons/mêmes arguments.

Enfin écrivez-le dans un .wav. Si tout se passe bien, le signal final n'a pas été modifié. **(avant il faut remettre le signal entre -1 et 1)**

Pour écrire dans un fichier wav :
**import soundfile as sf**
sf.write(""resultat.wav", sig_modif, sr)

Votre programme aura donc la structure suivante :

- Fourier (2.)
  - Spectre d'amplitude (3.)
  - Spectre de phase (3.)
- Traitement sur le spectre d'amplitude (6., 7.)
- Reconstruction du spectre (5.)
- Fourier inverse (2.)

## 3. Spectre d'amplitude et Spectre de phase

Un spectre (à un instant donné) est un tableau de nombre complexe (*a+ib*). Un nombre complexe peut aussi s'écrire.

$$X_k(\omega) = |X_k(\omega)|e^{j\phi_k} \approx amplitude * phase$$

Ce qui nous intéresse ici est le **spectre d'amplitude** (qui peut servir pour calculer un spectrogramme). Nous le modifierons puis nous réutiliserons le spectre de phase laissé intact.

Utilisez la fonction **librosa.magphase** pour obtenir ces deux spectres.

## 4. (Pause sur le débruitage)

*Rappel* : Pour un spectrogramme (la représentation du son par le spectre d'amplitude) on calcule normalement le spectre d'amplitude en prenant le logarithme (multiplié par 20) du module, cela revient à : $Spectre_amplitude[k] = 20.log_{10}(|X_k|)$

Vous pouvez afficher un spectrogramme :

Pour afficher vous pouvez utiliser la fonction imshow(…) de la librairie Matlibplot. Pour l'affichage, aspect=auto (cmaps = '..' pour choisir une palette de gris ou de couleurs spécifiques)

## 5. Reconstruction du spectre

Reconstruire un spectre complexe à partir du module et de la phase : Il suffit de multiplier les spectres d'amplitude et de phase.

*Rappel* : il faut utiliser le spectre d'amplitude sans le log (et non transposé).

Vérifiez que vous obtenez le signal initial.

## 6. Estimation du spectre d'amplitude du bruit :

a. Cela consiste à réaliser la moyenne sur les 4-8 premiers spectres (coefficients par coefficients) et à retourner un spectre d'amplitude moyenne (du bruit).
b. Pour réaliser le calcul, **il faudra d'abord transposer le tableau de stockage des spectres (.transpose()).** Ensuite, on peut directement utiliser la fonction numpy.mean(Spectre_amplitude_transposé[ :8], 0) (si les spectres sont placés ligne par ligne avec 1 ligne = 1 spectre d'amplitude).

Ce principe d'estimation du bruit suppose bien sûr que la parole ne commence pas pendant les premiers instants du signal (seul le bruit est présent).
[Prenez les fichiers *_bruit_ndB.wav]

## 7. Débruitage par soustraction spectrale

Réalisez une fonction de calcul de **soustraction spectrale** des spectres d'amplitude : elle soustrait au spectre d'amplitude le spectre de l'estimation du bruit.
La formule généralisée[1] de la soustraction spectrale est la suivante :

$$\hat{Y}_k = \left(\hat{X}_k^{\alpha} - \beta \hat{B}_k^{\alpha}\right)^{1/\alpha} si \text{ la différence } > 0$$
$$\hat{Y}_k = \gamma \hat{B}_k \qquad sinon$$

avec $X_k$ le spectre d'amplitude à l'instant $k$, $B_k$ le spectre d'amplitude du bruit estimé (à l'instant $k$, mais qui est ici le même pour tous les $k$) et $Y_k$ est le spectre d'amplitude résultat (à l'instant $k$).
Dans un premier temps, nous prendrons puissances $\alpha$=2 et $\beta$=1, et le coefficient $\gamma$=0.

Une fois le spectre d'amplitude « nettoyé », en le recombinant avec le spectre de phase, vous obtenez le signal débruité.

Essayez de deviner la signification des paramètres :
- $\beta$ représente quoi ?
- $\gamma$ sert à quoi ?

Pour vous aider, écoutez le résultat et jouez un peu sur les paramètres $\beta$ et $\gamma$ (du tableau ci-dessous).

[1] Formule un peu différente de l'article original [Berouti et al.]

| | | Ecoute (qualité ? bruit initial ? bruit parasite ?...) | | |
|---|---|---|---|---|
| $\beta$ | $\gamma$ | ***SNR 10dB*** | ***SNR 0 dB*** | ***SNR -10dB*** |
| 1 | 0 | | | |
| 1 | 0.01 | | | |
| 1 | 0.1 | | | |
| 1 | 0.25 | | | |
| 1 | 0.5 | | | |
| 5 | 0 | | | |
| 5 | 0.01 | | | |
| 5 | 0.25 | | | |
| 10 | 0 | | | |
| 10 | 0.01 | | | |
| 10 | 0.1 | | | |

| | | | | |
|---|---|---|---|---|
| 30 | 0 | | | |
| 30 | 0.01 | | | |
| 30 | 0.1 | | | |

## RENDU pour le Mercredi 29 avril (inclus)

Réalisez un mini-rapport :

[L'idée générale du rapport est de suivre une démarche scientifique : J'ai un algo à tester pour résoudre un problème, je présente l'algo, je mets en place des tests et j'évalue les résultats et je conclus sur l'algo]

- Réalisez un **diagramme/schéma** qui présente les différentes étapes de la soustraction spectrale (on part d'un signal, on le fenêtre, puis….)
- Présentez **différents tests** avec une brève analyse des résultats obtenus pour chacun suivant différents points de vue. Exemple : qualité du signal-résultat et qualité du débruitage (c-a-d est-ce que la parole est bien débruitée et aussi toujours intelligible…). Vous pourrez conclure de manière globale (y–a-t-il une combinaison optimale, ou un compromis, ou certains cas de bruit devrait mieux fonctionner… ) : Faites des tests pour 2 valeurs de $\alpha$ ( 1 et 2), pour 3-4 valeurs de $\beta$ (ex : 1,5,10,30) et pour 3-4 valeurs de $\gamma$ (ex : 0,0.01, 0.1, 0.25), sur les différents fichiers bruités.
- Il est possible bien sûr d'y inclure les spectrogrammes des fichiers débruités (avec la fonction savefig, ou sur l'interface de sortie des figures générées).

***Pour aller plus loin (on ne le demande pas pour le rapport):*** On aurait pu adapter la soustraction au niveau de bruit. Dans les travaux originaux, ils proposent de la formule suivante :

$$\hat{Y}_k = \left(\hat{X}_k - \beta \hat{B}_k\right) \quad si \text{ la différence } > \gamma \hat{B}_k$$
$$\hat{Y}_k = \gamma \hat{B}_k \qquad sinon$$

avec $\beta = \beta_0 - \frac{3}{20} SNR$ . où $\beta_0$ est la valeur adéquate pour un SNR de 0 dB. Vous pouvez tester avec un $\beta_0$ à 4 et testez cette formule pour les 3 fichiers (10, 0 et -10 dB). [pour $\gamma$ on pourra prendre 0.01]